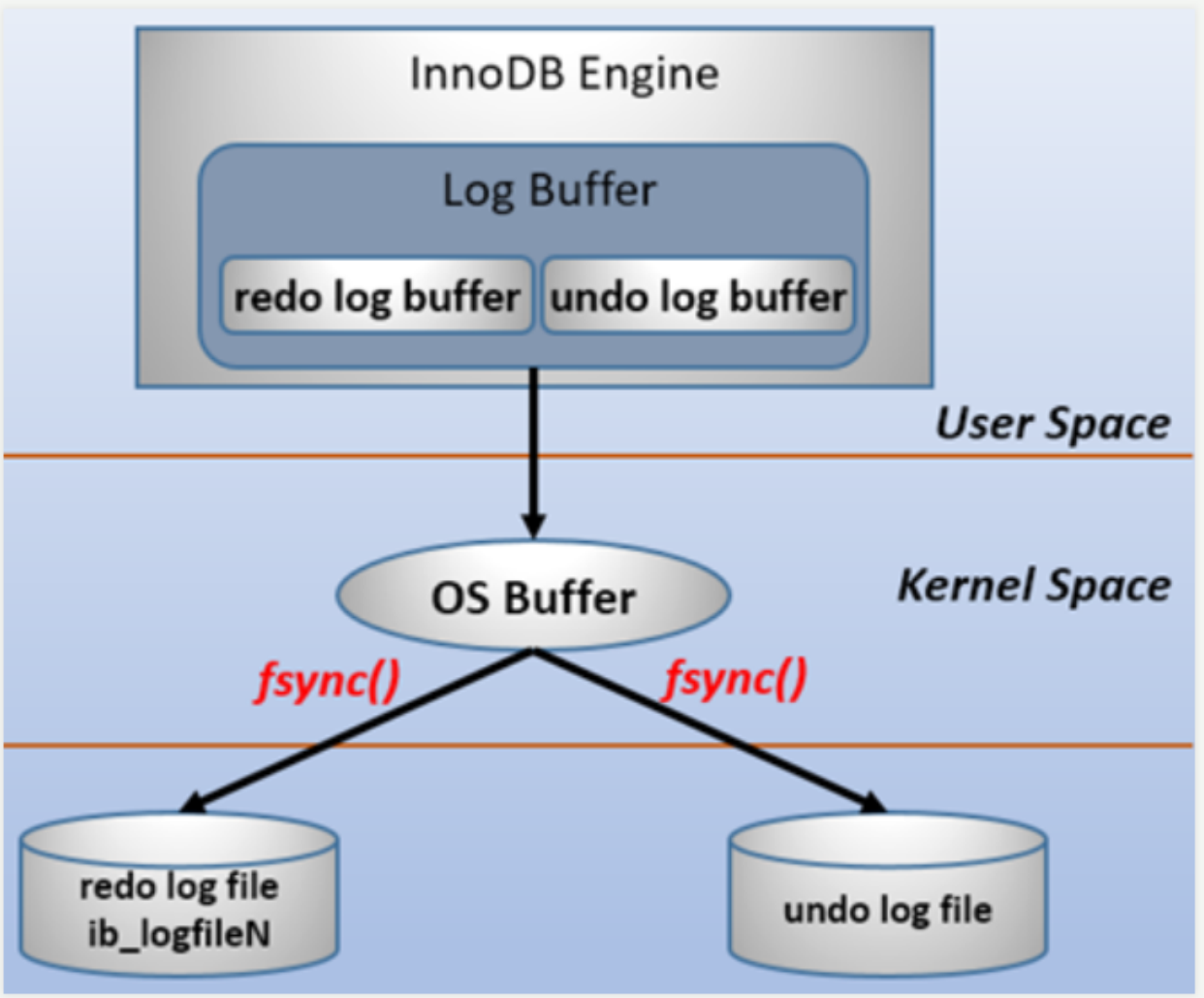
1.redo log通常是物理日志，记录的是数据页的物理修改，而不是某一行或某几行修改成怎样怎样，它用来恢复提交后的物理数据页(恢复数据页，且只能恢复到最后一次提交的位置)。

2.undo用来回滚行记录到某个版本。undo log一般是逻辑日志，根据每行记录进行记录。

## redo log：

只记录了哪一页修改了啥，具有幂等性。

包含两部分，一是redo log buffer，日志缓冲区，而是redo log file，这是物理层的，持久的。在事务进行时，必须首先把事务的所有事务日志写入到redo log file，因为如果出现宕机，需要根据这个事务日志进行更新本地磁盘的file。




# 事务commit>>>>redo log Buffer>>>>>>OS Buffer>>>>redo log file

其实中间是经历了2次缓存的，因为mysql是工作在用户空间的，用户空间的buffer要写入到磁盘的file，要先经过操作系统的内核空间的缓存os buffer，然后调用操作系统的fsync()操作，将os buffer的数据更新到磁盘的file。

另外，之所以要经过一层os buffer，是因为open日志文件的时候，open没有使用O_DIRECT标志位，该标志位意味着绕过操作系统层的os buffer，IO直写到底层存储设备。不使用该标志位意味着将日志进行缓冲，缓冲到了一定容量，或者显式fsync()才会将缓冲中的刷到存储设备。使用该标志位意味着每次都要发起系统调用。比如写abcde，不使用o_direct将只发起一次系统调用，使用o_object将发起5次系统调用。

直接复制了，不太理解这个标志位，反正就是加了某个标志位，每次将用户空间的缓存直接写入到磁盘，每调一次就是一次fsync()，而不用这个标志位，就是再加一层缓冲区，只需一次fsync()操作就可以将os buffer全部数据更新到磁盘。

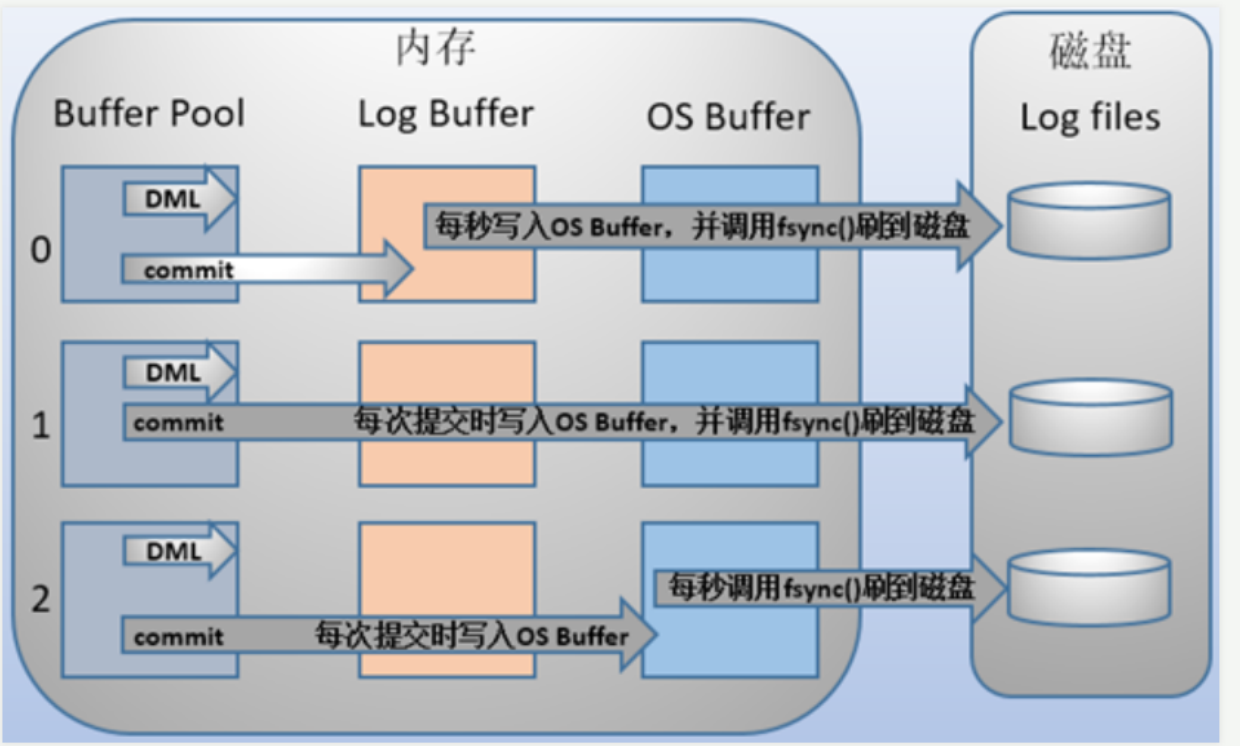
在commit的时候，我们可以自定义一个属性innodb_flush_log_at_trx_commit，值可以是0、1、2。默认是1。

0：commit的时候，log buffer的东西不会直接提交到os buffer或者磁盘，而是每秒从log buffer取一次，取到os buffer，并且同时调用fsync()到真正的磁盘里面。（每次commit数据到了log buffer，这是用户空间的缓冲区，mysql宕机了就GG了。此时用户空间的缓冲区的东西就没了，但是每秒会有一次，所以最多就是某一秒的log buffer的数据丢了。）

1：commit的时候，log buffer的东西直接先到os buffer，并且同时调用fsync()到真正的磁盘里面。

2：commit的时候，log buffer的东西直接先到os buffer，然后每秒从os buffer取数据，并且同时调用fsync()到真正的磁盘里面。（每次commit数据到了os buffer，mysql宕机了没问题，因为数据已经到了操作系统的缓冲区，除非操作系统宕机。）

Buffer Pool详解见另一篇文章




还有个变量innodb_flush_log_at_timeout，注意这个变量不是控制上面所说的从log buffer或者os buffer把数据刷新到磁盘的时间。看文档好像就是控制的这个时间，这个博客说不是，我看官方文档好像也是，有待商榷。

测试下来（我没测试），0最快，2还行，1最慢，但是2比0安全很多。
当然了1是最安全的，如果数据要求非常高，可以选择1。选择1的情况，需要控制的是事务的commit的次数，这样的话，就没问题了。（终于知道为什么要尽量减少事务提交次数了，博大精深，唉。）

## log block：

在innodb引擎中，redo log是以块进行存储的，每块512字节，称为redo log block。由三部分组成：日志块头、日志块尾、日志块主体。其实头占12字节，尾占8字节，主体占492字节。

redo log是用来记录数据页的变化，当变化超过492字节之后，就要用多个块进行存储。

日志块头包含4部分：

log_block_hdr_no(4字节)：记录了在redo log buffer的位置ID

log_block_hdr_data_length(2字节)：已记录的log大小，满了应该是0x200，512字节

log_block_first_rec_group(2字节)：表示该log block里面的第一个完整的log，也就是第一个日志主体的偏移量，因为每次变化可能占了多个log block。

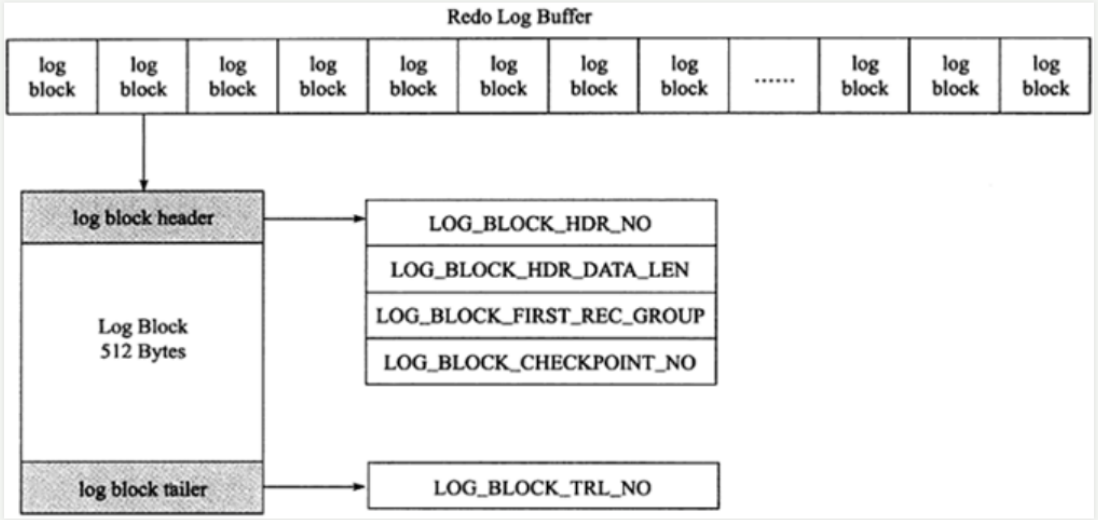
比如某次数据也的变化有600字节，其中600-512=88，这88个字节就会到下一个log block，那么如果该偏移量就是88+12=100，这个log block从100字节以后才是一个新的变化存储的值，88是上次的，12是头。如果等于512，相当于这个log block里面的东西全是上一个的。

log_block_checkpoint_no(4字节)：写入检查点信息的位置

日志块尾包含1部分：

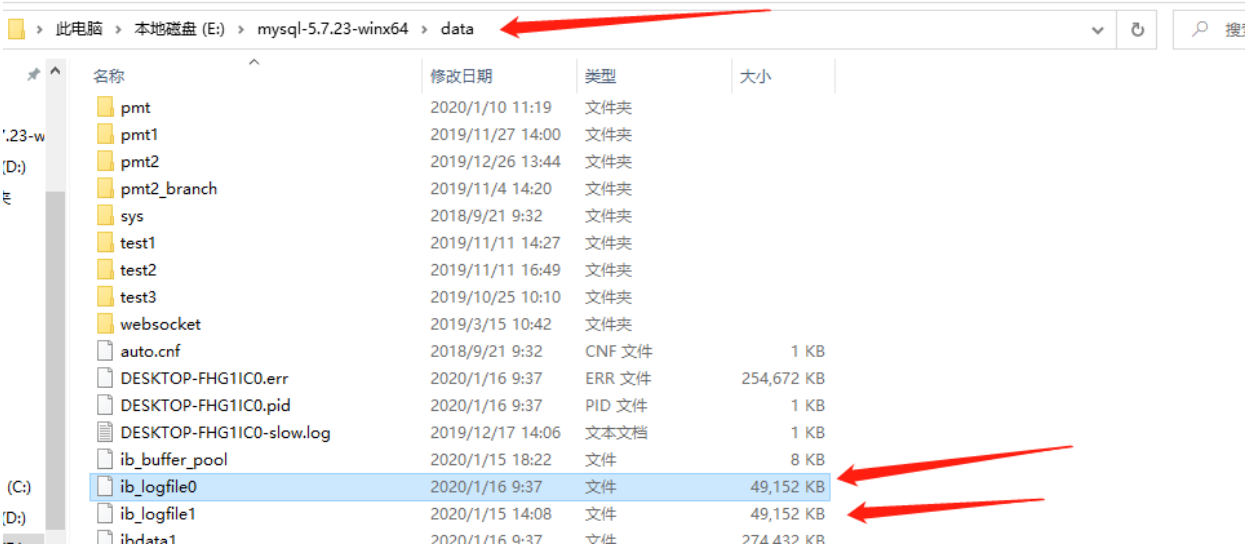
log_block_trl_no：记录了在redo log buffer的位置ID

上面所说的是一个日志块的内容，在redo log buffer或者redo log file on disk中，由很多log block组成。如下图：

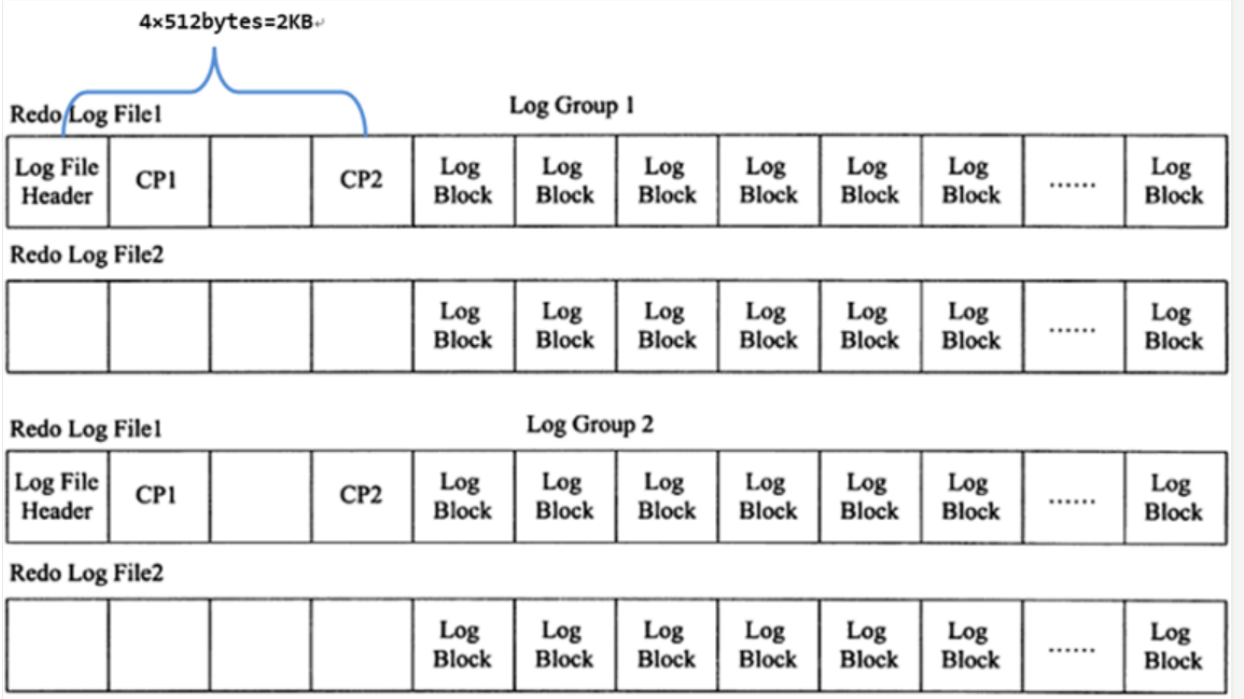



## log group和redo log file：

log group其实就是redo log group，里面默认有2个redo log file，这个redo log group是虚拟的，redo log file是磁盘实际存在的。




默认的redo log file的位置在这，将log buffer的log block刷新到log file，所以log file里面保存日志的方式也是log block的方式。不停的往尾部追加，第一个满了往第二个里面写，第二个满了会清空第一个file中的一部分，然后继续写。




在每一组也就是每个redo log group中的第一个file会腾出2KB记录4个特定的部分，之后才是log block，同一组其他的file，只会留出2KB，不记录。

log file设置的太小，就会导致不停的切换这2个file，太大恢复的时候就太慢。所有的事务commit，必须保证一层持久化才算成功。事务commit之后，首先在log file有了记录，然后更新pool的那一页的数据，就算成功了。pool是在内存的，如果没来得及更新到磁盘，数据库宕机了，重启的时候，靠log file恢复。当然，这个还是大点好，哪怕恢复的慢，正常操作就会快了。

对于数据库中的页的修改，首先修改在缓冲池中的页，然后再以一定的频率刷新到磁盘上，页冲缓冲池刷新回磁盘的操作并不是在每次页发送更新时触发，而是通过一种称为Checkpoint 的机制刷新回磁盘。这是为了提高数据库的整体性能。

## redo log的格式：

innodb存储数据的单元格式是页，所以在redo log里面也是以页的形式来记录的。

其中log block中492字节的部分是log body，该log body的格式分为4部分：

- redo_log_type：占用1个字节，表示redo log的日志类型。
- space：表示表空间的ID，采用压缩的方式后，占用的空间可能小于4字节。
- page_no：表示页的偏移量，同样是压缩过的。
- redo_log_body表示每个重做日志的数据部分，恢复时会调用相应的函数进行解析。例如insert语句和delete语句写入redo log的内容是不一样的。

如下图，分别是insert和delete大致的记录方式。

MLOG_REC_INSERT

| type | space | page_no | cur rec _offset | len & extra_info | into_bits | origin_ offset | mis_matc h_index | rec body |
|---|---|---|---|---|---|---|---|---|

MLOG_REC_DELETE

| type | space | page_no | offset |
|---|---|---|---|

## 日志刷盘的规则：

log buffer中未刷到磁盘的叫做脏日志。

触发脏日志刷新到磁盘的规则：

当innodb_flush_log_at_trx_commit值为1的时候，每次commit都会刷新到磁盘

每秒的刷新，由innodb_flush_log_at_timeout控制时间，默认是1。

当log buffeer使用的内存超过一半的时候

checkpoint的时候

其实如果innodb_flush_log_at_trx_commit值为1，触发脏日志只会是，每次commit和checkpoint

## 数据页刷盘的规则：

buffer pool中未刷新到磁盘的数据称为脏数据，数据和日志都已页的形式存在，所以统称脏页，又分为脏日志和脏数据。脏数据刷盘的规则只有一个，就是触发checkpoint。

## chckpoint：

innodb中的checkpoint非为2种，全部和部分：

sharp checkpoint：关闭数据库的时候，将所有脏页刷新到磁盘，由innodb_fast_shutdown参数控制，默认是1。

innodb_fast_shutdown是0的时候，当innodb关闭的时候，需要purge all，merge insert buffer，flush dirty pages，但是重启很快，其实mysql保证了所有的redo log file都要刷新到磁盘，要么在关闭的时候做，要么在重启的时候做。

innodb_fast_shutdown是1的时候，当innodb关闭的时候，只需要flush dirty pages。

innodb_fast_shutdown是2的时候，当innodb关闭的时候，只需要把log buffer的数据刷到redo log file，这一步其实很快，因为一直有线程在做，但是重启会比较慢。

purge all：主要是删除无用的undo页，

merge insert buffer：在非主键索引，非唯一索引的情况下，

flush dirty pages：刷新脏页到磁盘，也就是内存里面的脏数据到磁盘（这个磁盘指的是数据真正保存在磁盘的文件），log bufer里面的脏日志到redo log file。

fuzzy checkpoint：一次刷新一小部分的脏页，由以下几种情况触发：

master thread checkpoint：由master线程控制，每秒或每10秒刷入一定比率的脏数据到磁盘。

flush_lru_list checkpoint：innodb要保证lru列表中有100页左右的空闲页，如果没有，就会淘汰lru列表的旧列表尾端的页，但是要检查尾端的页有没有脏页，有的话就要强制触发check point，而检查的线程就是page cleaner thread，可以通过innodb_page_cleaners这个参数配置page cleaner的线程的个数，默认是1。

async/sync flush checkpoint：当重做日志，也就是redo log file不够用的时候，这时候也会触发check point，如果很多，就是同步刷新，不是很多，就异步刷新，很少就不刷新。

dirty page too much checkpoint：脏页数据太多，强制触发check point，比率是由innodb_max_dirty_pages_pct控制的，默认是75，就是75%。

## lsn：

称为日志的逻辑序列号，